

Disjoint Set Union

September 7, 2026

A (ID 2200110)

B (ID 2200110)

C (ID 2200110)

Department of Computer Science and Engineering

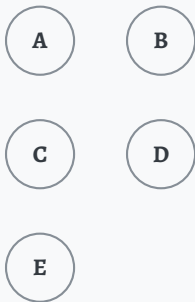
Outline

Introduction

Naive Implementation

Introduction

A Familiar Problem



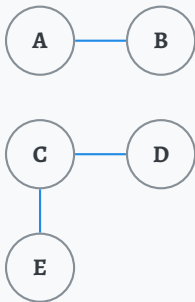
- Suppose there are five students in a classroom.
- At first, none of them know one another.

A Familiar Problem



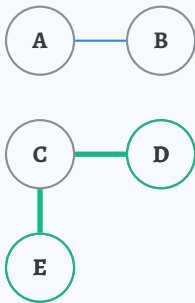
- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.

A Familiar Problem



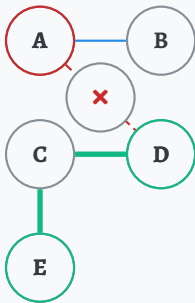
- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.
- Separately, C become friends with D and E.

A Familiar Problem



- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.
- Separately, C become friends with D and E.
- **Are D and E friends?** Yes — indirectly, through C, they belong to same friend circle.

A Familiar Problem



- Suppose there are five students in a classroom.
- At first, none of them know one another.
- A and B become friends.
- Separately, C become friends with D and E.
- **Are D and E friends?** Yes — indirectly, through C, they belong to same friend circle.
- What about **A and D**? No — they belong to two different circles entirely.

Larger Networks

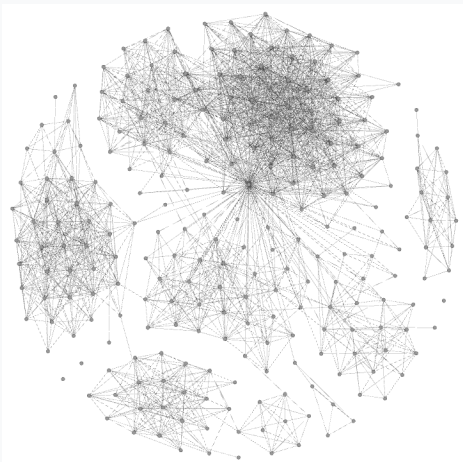


Figure: *Social Media Network Visualization*^a

- Consider a large social network, or equally, a large computer network, with countless members.

^a Pinterest: Visualise Your Facebook Friend Network

Larger Networks

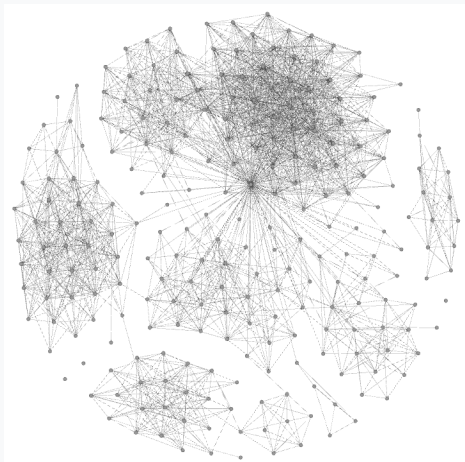


Figure: *Social Media Network Visualization*^a

- Consider a large social network, or equally, a large computer network, with countless members.
- We need to answer **efficiently**: **are any two given members connected?**

^aPinterest: Visualise Your Facebook Friend Network

Larger Networks

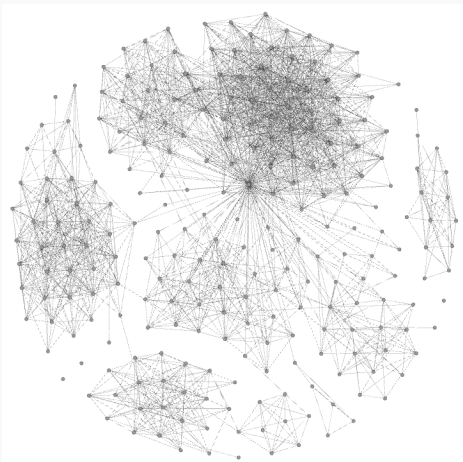


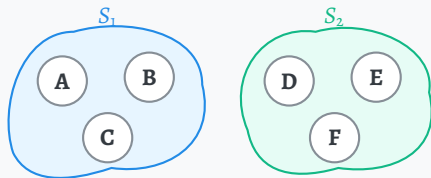
Figure: Social Media Network Visualization^a

- Consider a large social network, or equally, a large computer network, with countless members.
- We need to answer **efficiently**: **are any two given members connected?**

This is exactly what *Disjoint Set Union* answers.

^aPinterest: Visualise Your Facebook Friend Network

What Is a Disjoint Set Union?



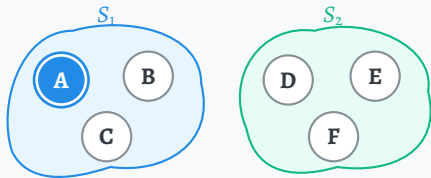
Definition

- Sets are **disjoint** if they share no elements:

$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

What Is a Disjoint Set Union?



Here,

- A is the representative of S_1

Definition

- Sets are **disjoint** if they share no elements:

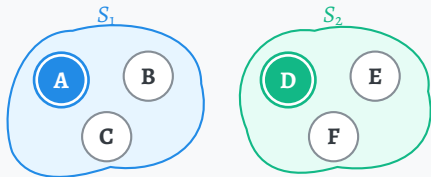
$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

Representative (“leader”)

- One chosen element identifies the entire set.

What Is a Disjoint Set Union?



Here,

- A is the representative of S_1
- D is the representative of S_2

Definition

- Sets are **disjoint** if they share no elements:

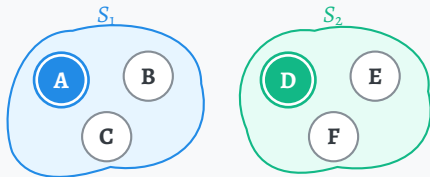
$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

Representative (“leader”)

- One chosen element identifies the entire set.

What Is a Disjoint Set Union?



Here,

- A is the representative of S_1
- D is the representative of S_2

Definition

- Sets are **disjoint** if they share no elements:

$$S_i \cap S_j = \emptyset \quad \text{for } i \neq j.$$

- **Disjoint Set Union** or **DSU** are a collection of such disjoint groups.

Representative (“leader”)

- One chosen element identifies the entire set.

Same representative $\iff$ same set.

The Interface

1. `make_set(x)`
 - Creates a new set whose only member is `x`.

The Interface

1. `make_set(x)`
 - Creates a new set whose only member is `x`.
2. `find(x)`
 - Returns the representative of the set containing `x`.

The Interface

1. `make_set(x)`
 - Creates a new set whose only member is `x`.
2. `find(x)`
 - Returns the representative of the set containing `x`.
3. `union(x, y)`
 - Merges the sets containing `x` and `y` into one.

The Interface

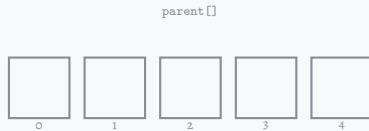
1. `make_set(x)`
 - Creates a new set whose only member is `x`.
2. `find(x)`
 - Returns the representative of the set containing `x`.
3. `union(x, y)`
 - Merges the sets containing `x` and `y` into one.

Because of the two main operations, `union` and `find`, this structure is also called **Union-Find**.

Naive Implementation

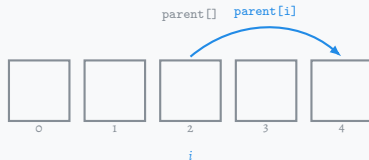
The Setup

- Maintain an array `parent []`



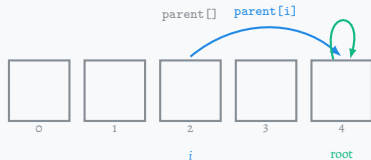
The Setup

- Maintain an array `parent[]`
- `parent[i]` stores the index of element i 's **immediate parent** — not necessarily its representative.



The Setup

- Maintain an array `parent[]`
- `parent[i]` stores the index of element i 's **immediate parent** — not necessarily its representative.
- Following parents eventually reaches the **root**, which is exactly the representative: a root points to itself, `parent[root] = root`.



make_set

Pseudocode

```
1  make_set(node):  
2      parent[node] ← node
```

Call once for each new element.

make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Parent array

index	0	1	2	3	4	5	6
parent	–	–	–	–	–	–	–

– means not initialized yet.

DSU Forest

No sets created yet.

make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

`make_set(0): parent[0] = 0`

Parent array

index	0	1	2	3	4	5	6
parent	0	–	–	–	–	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	–	–	–	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	–	–	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	–	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

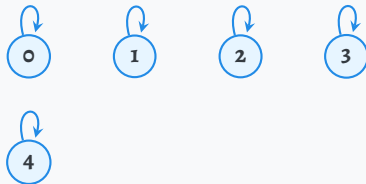
```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3  
make_set(4): parent[4] = 4
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	–	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

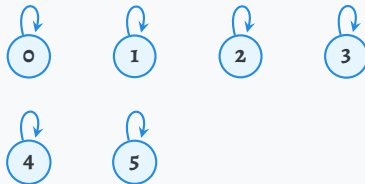
```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3  
make_set(4): parent[4] = 4  
make_set(5): parent[5] = 5
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	–

– means not initialized yet.

DSU Forest



make_set

Pseudocode

```
1 make_set(node):  
2   parent[node] ← node
```

Call once for each new element.

Operations

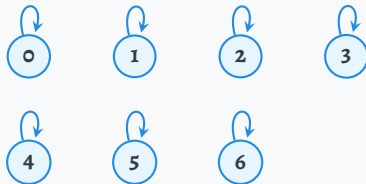
```
make_set(0): parent[0] = 0  
make_set(1): parent[1] = 1  
make_set(2): parent[2] = 2  
make_set(3): parent[3] = 3  
make_set(4): parent[4] = 4  
make_set(5): parent[5] = 5  
make_set(6): parent[6] = 6
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

– means not initialized yet.

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

find: Follow Parents to the Root

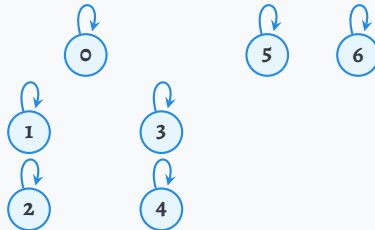
Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   return find(parent[node])
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2     if parent[node] = node then  
3         return node  
4     return find(parent[node])
```

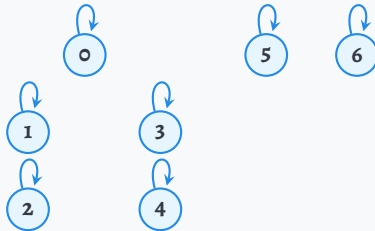
Operations

find(4)

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   return find(parent[node])
```

Operations

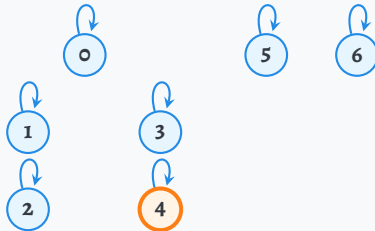
find(4)

- Read `parent[4] = 4`.

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



find: Follow Parents to the Root

Pseudocode

```
1 find(node):  
2   if parent[node] = node then  
3     return node  
4   return find(parent[node])
```

Operations

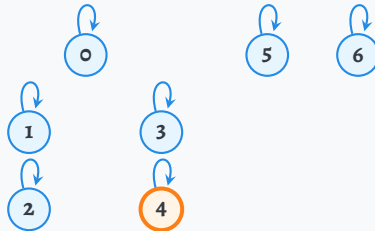
find(4)

- Read `parent[4] = 4`.
- The node is its own parent: return 4.

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

union: Find Both Roots, Then Link

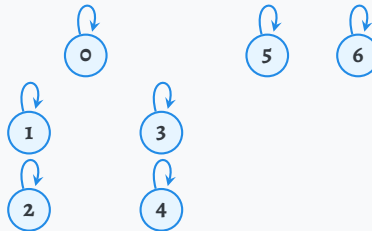
Pseudocode

```
1 union(first, second):  
2   rootA ← find(first)  
3   rootB ← find(second)  
4   if rootA ≠ rootB then  
5     parent[rootB] ← rootA
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



Pseudocode

```
1 union(first, second):
2     rootA ← find(first)
3     rootB ← find(second)
4     if rootA ≠ rootB then
5         parent[rootB] ← rootA
```

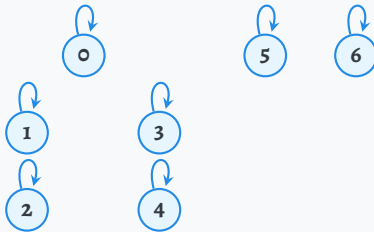
Operations

```
union(1, 2)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

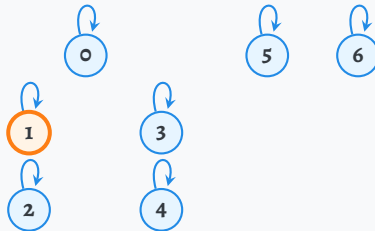
union(1, 2)

find(1): parent[1] = 1 $\Rightarrow$ rootA = 1

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

union(1, 2)

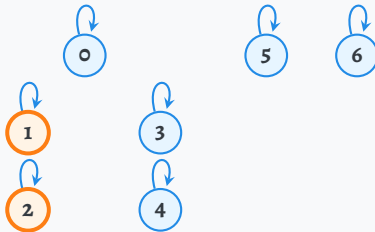
find(1): parent[1] = 1 ⇒ rootA = 1

find(2): parent[2] = 2 ⇒ rootB = 2

Parent array

index	0	1	2	3	4	5	6
parent	0	1	2	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

union(1, 2) parent[2] ← 1

find(1): parent[1] = 1 ⇒ rootA = 1

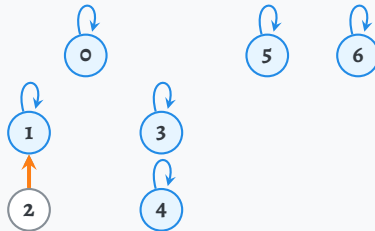
find(2): parent[2] = 2 ⇒ rootB = 2

Different roots: attach 2 under 1.

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

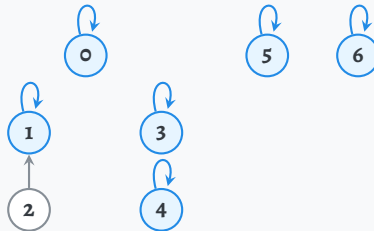
Operations

```
union(1, 2)  parent[2] ← 1  
union(0, 1)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2   rootA ← find(first)  
3   rootB ← find(second)  
4   if rootA ≠ rootB then  
5     parent[rootB] ← rootA
```

Operations

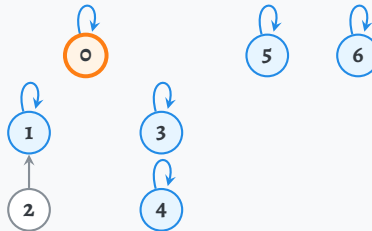
```
union(1, 2)  parent[2] ← 1  
union(0, 1)
```

```
find(0): parent[0] = 0 ⇒ rootA = 0
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

Operations

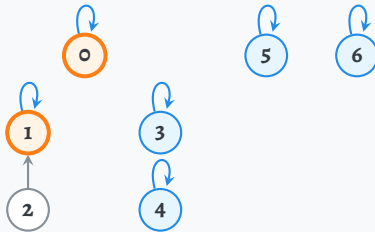
```
union(1, 2)  parent[2] ← 1  
union(0, 1)
```

```
find(0): parent[0] = 0 ⇒ rootA = 0  
find(1): parent[1] = 1 ⇒ rootB = 1
```

Parent array

index	0	1	2	3	4	5	6
parent	0	1	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

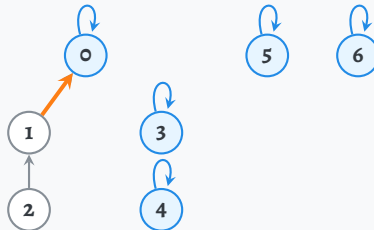
union(1, 2) parent[2] ← 1
union(0, 1) parent[1] ← 0

find(0): parent[0] = 0 ⇒ rootA = 0
find(1): parent[1] = 1 ⇒ rootB = 1
Different roots: attach 1 under 0.

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

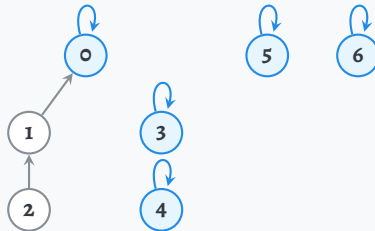
Operations

```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

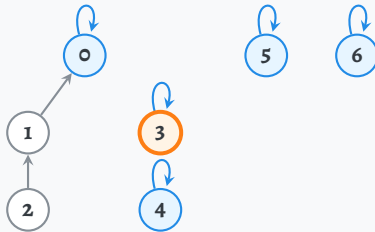
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)
```

```
find(3): parent[3] = 3 ⇒ rootA = 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

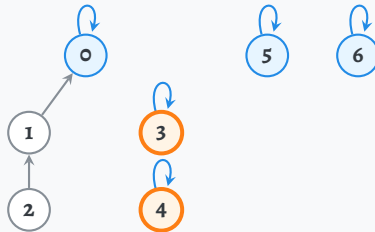
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)
```

```
find(3): parent[3] = 3 ⇒ rootA = 3  
find(4): parent[4] = 4 ⇒ rootB = 4
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	4	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

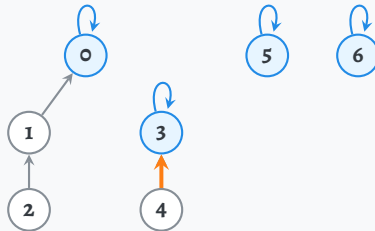
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3
```

```
find(3): parent[3] = 3 ⇒ rootA = 3  
find(4): parent[4] = 4 ⇒ rootB = 4  
Different roots: attach 4 under 3.
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

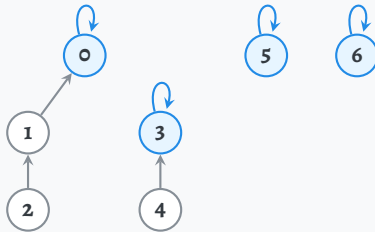
Operations

```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

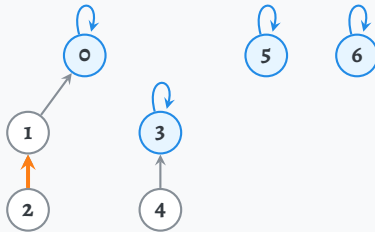
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

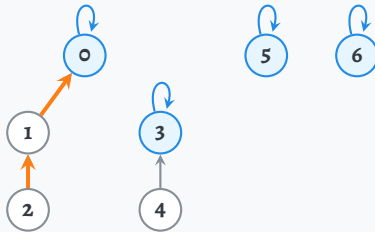
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1 → 0
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

Operations

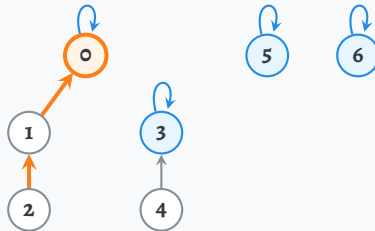
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

find(2): $2 \rightarrow 1 \rightarrow 0$; $\text{parent}[0] = 0 \Rightarrow \text{rootA} = 0$

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1  union(first, second):  
2      rootA ← find(first)  
3      rootB ← find(second)  
4      if rootA ≠ rootB then  
5          parent[rootB] ← rootA
```

Operations

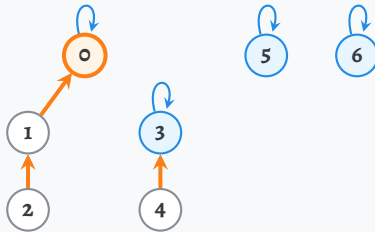
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1 → 0; parent[0] = 0 ⇒ rootA = 0  
find(4): 4 → 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

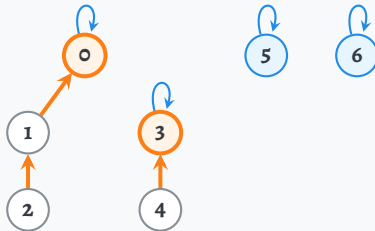
```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)
```

```
find(2): 2 → 1 → 0; parent[0] = 0 ⇒ rootA = 0  
find(4): 4 → 3; parent[3] = 3 ⇒ rootB = 3
```

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	3	3	5	6

DSU Forest



union: Find Both Roots, Then Link

Pseudocode

```
1 union(first, second):  
2     rootA ← find(first)  
3     rootB ← find(second)  
4     if rootA ≠ rootB then  
5         parent[rootB] ← rootA
```

Operations

```
union(1, 2)  parent[2] ← 1  
union(0, 1)  parent[1] ← 0  
union(3, 4)  parent[4] ← 3  
union(2, 4)  parent[3] ← 0
```

find(2): $2 \rightarrow 1 \rightarrow 0$; $\text{parent}[0] = 0 \Rightarrow \text{rootA} = 0$
find(4): $4 \rightarrow 3$; $\text{parent}[3] = 3 \Rightarrow \text{rootB} = 3$
Different roots: attach 3 under 0, not 4 under 2.

Parent array

index	0	1	2	3	4	5	6
parent	0	0	1	0	3	5	6

DSU Forest

